3-3

A2A 프로토콜의 이해

학습 목차

- **Agent 2 Agent**
  - A2A 등장 배경
  - MCP와 A2A의 차이
- **A2A 핵심 개념**
  - Agent Card
  - Task, Message, Artifact
- **A2A Server 구현**
  - AgentExecutor
  - AgentCard
  - RequestHandler
- **A2A Client 구현**
  - A2ACardResolver
  - SendMessageRequest

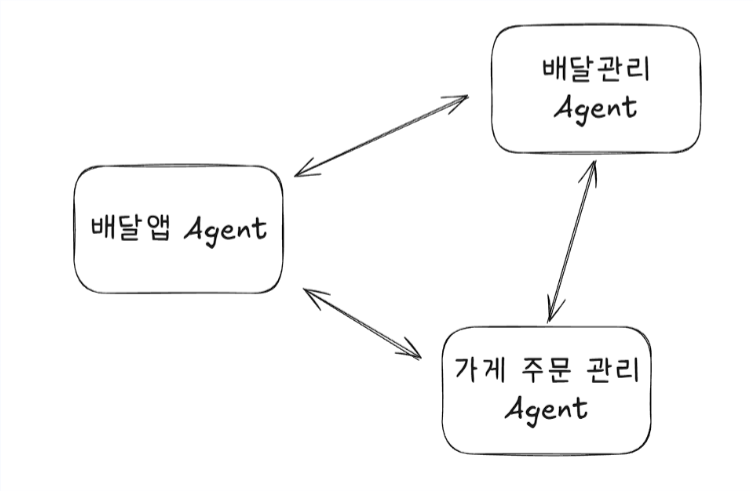
© Scopelabs. All Rights Reserved

Agent 2 Agent

A2A 등장 배경

A2A는 Agent가 다른 Agent에게 작업을 위임하고 결과를 주고받기 위한 통신 관점의 프로토콜이다.

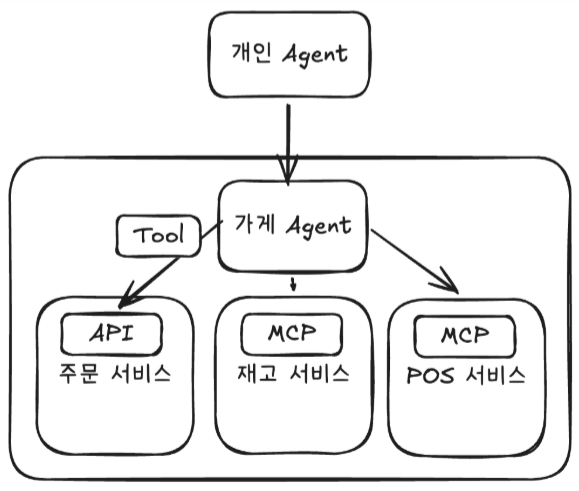
- Multi-Agent 시스템에서는 Agent마다 역할, 도구, 실행 환경이 다를 수 있다.
- 서로 다른 프레임워크로 만든 Agent도 협업할 수 있어야 한다.
- 단순 함수 호출보다 긴 작업, 상태 변화, 결과 산출물을 다룰 수 있는 구조가 필요하다.
- 요청하는 Agent는 상대 Agent의 내부 구현을 몰라도 된다.



MCP와 A2A의 차이

MCP와 A2A는 모두 연결 표준이지만 연결 대상이 다르다.

- MCP는 Agent와 Tool 또는 데이터 소스를 연결한다.
- A2A는 Agent와 Agent를 연결한다.
- MCP의 핵심은 도구 사용이고, A2A의 핵심은 작업 위임과 협업이다.
- 전문 Agent는 A2A로 작업을 받고, 내부적으로 MCP 도구를 사용할 수 있다.



A2A 핵심 개념

Agent Card

Agent Card는 Agent가 자신이 누구이며 어떤 일을 할 수 있는지 외부에 설명하는 메타데이터이다.

- Agent의 이름, 설명, 제공 기능을 포함한다.
- 다른 Agent나 클라이언트가 협업 가능성을 판단하는 기준이 된다.
- 입출력 모드, 연결 정보, 제공 Skill을 함께 설명한다.

```
{
  "name": "Researcher Agent",
  "description": "주어진 주제를 웹에서 조사하고 요약 정보를 제공합니다.",
   // ...
  "skills": [
    {
      "id": "web_research",
      "name": "웹 리서치",
      "tags": ["research", "web"]
    }
  ]
}
```

Task, Message, Artifact

A2A의 협업 단위는 Task, Message, Artifact로 나누어 이해할 수 있다.

- Task: Agent에게 위임되는 작업의 단위
- Message: Agent 간 요청, 질문, 중간 응답을 담는 커뮤니케이션
- Artifact: 보고서, 코드, 파일, 구조화 데이터 같은 최종 또는 중간 산출물

긴 작업은 Task 상태와 Message 이력을 통해 추적한다.

A2A Server 구현

AgentExecutor

`AgentExecutor`: A2A 요청을 실제 Agent 실행으로 연결하는 서버의 핵심 클래스

- `RequestContext`에서 사용자 입력과 task/context id를 읽음
- 내부 LangGraph Agent를 실행
- 결과를 `EventQueue`에 `t.Message`로 발행

```
class ResearcherExecutor(AgentExecutor):
    async def execute(self, context, event_queue):
        user_text = context.get_user_input() or ""

        result = await langgraph_agent.ainvoke({
            "messages": [HumanMessage(content=user_text)]
        })
        answer = result["messages"][-1].content

        await event_queue.enqueue_event(t.Message(...))
```

AgentCard

`AgentCard`: 클라이언트가 Agent의 능력과 연결 정보를 발견하기 위한 메타데이터

- `/.well-known/agent-card.json`에서 JSON으로 노출
- 이름, 설명, 입출력 모드, 지원 인터페이스를 포함
- `skills`로 Agent가 제공하는 대표 기능을 설명

```
agent_card = t.AgentCard(
    name="Researcher Agent",
    description="주어진 주제를 웹에서 조사하고 요약 정보를 제공합니다.",
    # ...
    skills=[
        t.AgentSkill(
            #...
        )
    ],
)
```

RequestHandler

`RequestHandler`: A2A 서버의 라우팅 계층과 실행 계층을 연결하는 객체

- `agent_executor`에 실제 실행 로직을 연결
- `task_store`와 `queue_manager`로 작업 상태와 이벤트를 관리
- Agent Card 조회 라우트와 JSON-RPC 라우트를 Starlette 앱에 등록

```
request_handler = DefaultRequestHandlerV2(
    agent_executor=ResearcherExecutor(),
    task_store=task_store,
    agent_card=agent_card,
    queue_manager=queue_manager,
)

routes = [
    *create_agent_card_routes(agent_card),
    *create_jsonrpc_routes(request_handler, rpc_url="/"),
]
```

A2A Client 구현

A2ACardResolver

`A2ACardResolver`: 원격 Agent의 Agent Card를 읽어 통신 준비를 하는 도구

- `A2A-Version` 헤더로 SDK 1.x 요청임을 명시
- Agent Card를 먼저 읽어 URL, 기능, 입출력 모드를 확인
- Agent Card를 기반으로 JSON-RPC transport와 Client 생성

```
headers = {VERSION_HEADER: PROTOCOL_VERSION_1_0}

async with httpx.AsyncClient(headers=headers, timeout=60.0) as http:
    resolver = A2ACardResolver(http, RESEARCHER_URL)
    card = await resolver.get_agent_card()

    transport = JsonRpcTransport(http, card, f"{RESEARCHER_URL}/")
    client = BaseClient(card, ClientConfig(), transport, [])
```

SendMessageRequest

`SendMessageRequest`: 사용자 요청을 A2A `Message`로 감싸 원격 Agent에게 전송하는 요청 객체

- 사용자 입력은 `t.Message`의 `parts`에 텍스트로 담음
- `client.send_message(req)`는 응답 스트림을 반환
- 응답 메시지의 `parts`에서 Agent가 생성한 텍스트를 추출

```
req = t.SendMessageRequest(
    message=t.Message(
        role=t.Role.ROLE_USER,
        parts=[t.Part(text=query)],
        message_id=str(uuid.uuid4()),
    )
)

async for response in client.send_message(req):
    # ...
```